

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЁТ
к лабораторной работе №3
на тему
**«ОБРАБОТКА ТЕКСТОВОЙ ИНФОРМАЦИИ.
РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ»**
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Постановка задачи	3
2	Описание функционала скрипта	4
2.1	Архитектура скрипта	4
2.2	Обработка аргументов командной строки	4
2.3	Валидация входных данных	4
2.4	Агрегация и форматированный вывод результатов	4
	Вывод	5
	Список использованных источников	6
	Приложение А (обязательное) Листинг программного кода	7

1 ПОСТАНОВКА ЗАДАЧИ

Разработать скрипт (или скрипты) для обработки входных данных в виде CSV-файлов в соответствии с вариантом 4 («Статистика»). Скрипт должен использовать утилиту *awk* или другие утилиты обработки текста и предусматривать поведение при ошибочных или «неочищенных» входных данных. Входные данные представляют собой файл со списками товаров в формате: наименование, количество, цена. Одно и то же наименование может встречаться многократно. Выходные данные — список товаров без повторов с вычисленным суммарным количеством, средней ценой и общей стоимостью.

2 ОПИСАНИЕ ФУНКЦИОНАЛА СКРИПТА

2.1 Архитектура скрипта

Скрипт реализован в виде обертки на языке *Bash* [1] с вызовом утилиты *awk* [2] для основной обработки. Оболочка *Bash* отвечает за предварительные проверки файлов и аргументов командной строки, а *awk* выполняет построчный парсинг, валидацию и агрегацию табличных данных. Ошибки и предупреждения выводятся в стандартный поток ошибок (*stderr*), чтобы не смешиваться с полезным выводом результата.

2.2 Обработка аргументов командной строки

Проверка параметров запуска осуществляется в *Bash*. При отсутствии переданного имени файла или в случае отсутствия файла по указанному пути скрипт выводит информативное сообщение в поток ошибок и завершает работу с ненулевым кодом статуса *1*. Это предотвращает передачу некорректных аргументов обработчику.

2.3 Валидация входных данных

Обработка строк CSV-файла выполняется утилитой *awk*. Встроенная переменная *FS* (разделитель полей) устанавливается равной запятой в блоке *BEGIN*. При построчном чтении скрипт идентифицирует и игнорирует строку-заголовок, проверяя номер текущей записи (переменная *NR == 1*). Валидация формата строки проверяет наличие ровно трех непустых колонок (*NF != 3*). Затем выполняется проверка типов данных с помощью регулярных выражений: количество должно быть целым положительным числом, а цена — числом (в том числе с плавающей точкой). При обнаружении невалидных данных строка пропускается командой *next*, а в *stderr* выводится диагностическое предупреждение с указанием номера строки.

2.4 Агрегация и форматированный вывод результатов

Для накопления статистики применяются ассоциативные массивы (словари) [3] *total_qty* (суммарное количество) и *total_cost* (общая стоимость), где ключом выступает наименование товара. После успешного прочтения всего файла управление переходит к блоку *END*. В нём выводится отформатированная шапка таблицы. Расчет средней цены товара выполняется итеративно по всем накопленным ключам (с предусмотренной защитой от деления на ноль). Форматирование колонок с заданной шириной и выравниванием реализуется встроенной функцией *printf*.

ВЫВОД

В ходе выполнения лабораторной работы были изучены методы и средства обработки текстовой информации в UNIX-подобных операционных системах с использованием регулярных выражений и специализированной утилиты *awk*. Был разработан скрипт *stat_calc.sh*, способный извлекать данные из CSV-файла, отфильтровывать ошибочные, неполные или «грязные» записи и осуществлять вычисление агрегированной статистики по категориям товаров (суммарное количество, средняя цена, общая стоимость). На практике были закреплены навыки работы с ассоциативными массивами в *awk*, перенаправлением стандартных потоков ввода/вывода, а также валидацией аргументов в сценариях оболочки *Bash*.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Shell Scripting Tutorial [Электронный ресурс]. – Режим доступа: <https://www.shellscript.sh/>. – Дата доступа: 14.03.2026.

[2] AWK command in Linux [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/linux-unix/awk-command-unixlinux-examples/>. – Дата доступа: 14.03.2026.

[3] Array Basics in Shell Scripting [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/linux-unix/array-basics-shell-scripting-set-1/>. – Дата доступа: 14.03.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – stat_calc.sh

```
#!/bin/bash

if [ -z "$1" ]; then
    echo "Error: Input file not specified." >&2
    echo "Usage: $0 <path_to_file.csv>" >&2
    exit 1
fi

INPUT_FILE="$1"

if [ ! -f "$INPUT_FILE" ]; then
    echo "Error: File '$INPUT_FILE' not found." >&2
    exit 1
fi

awk '
BEGIN {
    FS = ","
}

{
    if (NR == 1 && $1 ~ /^[Нн]аименование/) next;

    if (NF != 3 || $1 == "" || $2 == "" || $3 == "") {
        print "Warning: Line " NR " has invalid format (skipped): " $0
        > "/dev/stderr"
        next
    }

    name = $1
    qty = $2
    price = $3

    if (qty !~ /^[0-9]+$/ || price !~ /^[0-9]+(\.[0-9]+)?$/) {
        print "Warning: Line " NR " contains invalid numbers
(skipped): " $0 > "/dev/stderr"
        next
    }

    total_qty[name] += qty
    total_cost[name] += (qty * price)
}

END {
    print ""
    printf "%-20s | %-12s | %-12s | %-15s\n", "Наименование", "Общ.
кол-во", "Ср. цена", "Общая стоимость"
    print
    "-----"
```

```
    for (name in total_qty) {
        if (total_qty[name] > 0) {
            avg_price = total_cost[name] / total_qty[name]
        } else {
            avg_price = 0
        }

        printf "%-20s | %-12d | %-12.2f | %-15.2f\n", name,
total_qty[name], avg_price, total_cost[name]
    }
}' "$INPUT_FILE"
```